

Competitive Programming

Section 55 Editorial: Randomization And Hashing

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Equal Substrings	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Palindrome Substrings	5
3.1	Restatement	5
3.2	Observations and algorithm	5
3.3	Correctness proof	5
3.4	Complexity	5
3.5	Full C++ solution	5
3.6	Full Python solution	5
4	C. Anagram Substrings	6
4.1	Restatement	6
4.2	Observations and algorithm	6
4.3	Correctness proof	6
4.4	Complexity	6
4.5	Full C++ solution	6
4.6	Full Python solution	6
5	D. Longest Repeat	7
5.1	Restatement and monotonicity	7
5.2	Hash check for one length	7
5.3	Algorithm	7
5.4	Correctness proof	7
5.5	Complexity and implementation notes	8
5.6	Full C++ solution	8
5.7	Full Python solution	9
5.8	Tests that matter	10
6	What to carry forward	11
7	Complete upsolve notes	12
7.1	A. Equal Substrings	12
7.2	B. Palindrome Substrings	12
7.3	C. Anagram Substrings	12

1 Editorial overview

1.1 Problem-specific proof map

A. Equal Substrings. Equal strings have equal normalized polynomial hashes; the local double-hash implementation makes accidental equality negligible, and all queries are answered from prefix hashes in $O(1)$.

B. Palindrome Substrings. A substring is palindromic precisely when its forward hash equals the aligned reverse-string hash. Normalization compares the same exponent range.

C. Anagram Substrings. Prefix counts for each letter make the count vector of any substring a subtraction. Two strings are anagrams exactly when all their letter counts agree.

1.2 Detailed solution guide

A. Substring Equality. The definition asks whether two substrings have the same characters in the same order. The local exact solution compares them directly. The scalable version stores prefix hashes so the hash of $[l, r]$ can be extracted in $O(1)$ and normalized by length. Equal hashes are treated as evidence of equality; double hashing reduces collision risk.

B. Palindrome Hashing. A substring is a palindrome iff it equals its reverse. With hashes, compare the forward hash of $[l, r]$ in s with the corresponding forward hash in $\text{reverse}(s)$. The direct local implementation compares mirrored characters, which proves the same condition without collision concerns.

C. Anagram Queries. Two substrings are anagrams iff each character appears the same number of times in both. Prefix frequency arrays let us compute the 26-count vector for any substring by subtraction. Comparing those vectors is an exact deterministic hash for lowercase letters.

Common pitfalls. Hash indices are easy to shift by one, so write the substring formula once and test length-one substrings. Randomized hashing is probabilistic; frequency vectors for small alphabets are exact.

The local exercises use exact checks when they are cheap and prefix summaries when they are better.

- A compares equal-length substrings by their stored representation.
- B compares mirrored characters, which is exact for the local constraints; the lesson shows how forward/reverse hashes replace it at scale.
- C uses 26 prefix frequency arrays, which is deterministic and collision-free for lowercase anagram queries.

Hashing is presented as an optimization path, not as a substitute for a proof of what is being compared.

2 A. Equal Substrings

2.1 Restatement

Answer whether two same-length substrings are equal.

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(c
```

2.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if s[l1-1:r1]==s[l2-1:r2])
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Palindrome Substrings

3.1 Restatement

For each query, say whether the first substring is a palindrome.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(
```

3.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if s[l1-1:r1]==s[l1-1:r2])
    print("\n".join(out))
if __name__=="__main__": main()
```

4 C. Anagram Substrings

4.1 Restatement

Answer whether two same-length substrings have the same multiset of letters.

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(
```

4.6 Full Python solution

```
import sys,collections
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if sorted(s[l1-1:r1])
    print("\n".join(out))
if __name__=="__main__": main()
```

5 D. Longest Repeat

5.1 Restatement and monotonicity

Find the maximum length L for which two different starting positions contain the same length- L substring. Overlap is allowed. Comparing every pair of substrings directly is cubic in the worst case.

Define `possible(L)` to mean that some length- L substring occurs twice. This predicate is monotone: if a repeated substring of length L exists, its prefix of every smaller positive length is also repeated. Therefore binary search can find the largest true length.

5.2 Hash check for one length

Precompute two polynomial prefix hashes and powers. For every start position, extract the pair of normalized hashes for its length- L substring and insert the pair into a set. A pair already in the set proves a repeated candidate.

For `banana` and $L=3$, the windows are `ban`, `ana`, `nan`, `ana`; the second `ana` produces the first duplicate. Since `possible(3)` is true and `possible(4)` is false, the answer is three.

5.3 Algorithm

```
precompute two prefix-hash arrays and two power arrays
binary search L from 1 through n:
    scan all length-L windows
    return true as soon as a double-hash pair repeats
print the largest L whose scan returned true
```

The two-modulus hash is probabilistic: a collision could make two unequal substrings look equal. With independent large prime moduli this risk is negligible for normal contest use. A suffix array is the deterministic alternative when the problem requires a proof without that assumption.

5.4 Correctness proof

Lemma 1. Ignoring hash collisions, `possible(L)` returns true exactly when a repeated length- L substring exists.

Proof. The scan inserts the hash of every length- L window. Equal substrings have equal normalized hash pairs, so a second occurrence is found. Conversely, if no pair repeats, no two windows are equal under collision-free hashing, so no repeated substring exists.

Lemma 2. `possible(L)` is monotone.

Proof. If two length- L substrings are equal, their first K characters are equal for every $1 \leq K \leq L$. Thus `possible(L)` implies `possible(K)`.

Theorem. Subject to the stated negligible collision probability, the algorithm prints the longest repeated-substring length.

Proof. Lemma 1 evaluates the predicate correctly. Lemma 2 makes its true lengths an initial interval, so binary search returns its largest member. That member is exactly the requested maximum length.

5.5 Complexity and implementation notes

One check scans $O(n)$ windows; binary search performs $O(\log n)$ checks, for $O(n \log n)$ expected time and $O(n)$ memory. Store hash pairs rather than only one modulus. Normalize a substring using the power for its **length**, not its right endpoint. The answer may be 0, and a repeated length n is impossible because there is only one whole-string window.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;
    const long long mod1 = 1'000'000'007, mod2 = 1'000'000'009, base = 911'382'323;
    int n = (int)s.size();
    vector<long long> power1(n + 1, 1), power2(n + 1, 1), hash1(n + 1), hash2(n + 1);
    for (int index = 1; index <= n; ++index) {
        power1[index] = power1[index - 1] * base % mod1;
        power2[index] = power2[index - 1] * base % mod2;
        hash1[index] = (hash1[index - 1] * base + s[index - 1]) % mod1;
        hash2[index] = (hash2[index - 1] * base + s[index - 1]) % mod2;
    }
    auto repeated = [&](int length) {
        unordered_set<unsigned long long> seen;
        seen.reserve(2 * (n - length + 1));
        for (int left = 0; left + length <= n; ++left) {
            int right = left + length;
            int first = (hash1[right] - hash1[left] * power1[length] % mod1 + mod1) % mod1;
            int second = (hash2[right] - hash2[left] * power2[length] % mod2 + mod2) % mod2;
            unsigned long long key = (unsigned long long)first * mod2 + second;
            if (!seen.insert(key).second) return true;
        }
    };
}
```

```

        return false;
};
int low = 1, high = n, answer = 0;
while (low <= high) {
    int middle = (low + high) / 2;
    if (repeated(middle)) answer = middle, low = middle + 1;
    else high = middle - 1;
}
cout << answer << '\n';
}

```

5.7 Full Python solution

```

import sys

MOD1 = 1_000_000_007
MOD2 = 1_000_000_009
BASE = 911_382_323

def main() -> None:
    s = sys.stdin.buffer.readline().strip()
    n = len(s)
    power1 = [1] * (n + 1)
    power2 = [1] * (n + 1)
    hash1 = [0] * (n + 1)
    hash2 = [0] * (n + 1)
    for index, byte in enumerate(s, 1):
        power1[index] = power1[index - 1] * BASE % MOD1
        power2[index] = power2[index - 1] * BASE % MOD2
        hash1[index] = (hash1[index - 1] * BASE + byte) % MOD1
        hash2[index] = (hash2[index - 1] * BASE + byte) % MOD2

    def repeated(length: int) -> bool:
        seen: set[tuple[int, int]] = set()
        for left in range(n - length + 1):
            right = left + length
            key = (
                (hash1[right] - hash1[left] * power1[length]) % MOD1,
                (hash2[right] - hash2[left] * power2[length]) % MOD2,
            )
            if key in seen:
                return True
            seen.add(key)
        return False

    low, high, answer = 1, n, 0

```

```
while low <= high:
    middle = (low + high) // 2
    if repeated(middle):
        answer = middle
        low = middle + 1
    else:
        high = middle - 1
print(answer)

if __name__ == "__main__":
    main()
```

5.8 Tests that matter

One character and all distinct characters: answer 0.
All equal characters: answer n-1.
Overlapping repetitions: banana and aaaaaa.
Two long equal blocks at different positions.
Near-equal windows that differ only at their final character.

The tiny random oracle compares real slices, not hashes. The scale case is a 200000-character uniform string, which makes every binary-search scan large.

6 What to carry forward

7 Complete upsolve notes

7.1 A. Equal Substrings

Build powers and prefix hashes. Normalize each same-length substring hash by subtracting its preceding prefix times the matching power; equal hashes answer YES probabilistically. The prefix recurrence proves the extraction formula. Preprocess $O(n)$, query $O(1)$; double hash in adversarial settings and test prefix/suffix/full ranges.

7.2 B. Palindrome Substrings

Build the same hash for s and reversed s . Compare $[l, r)$ against reversed $[n-r, n-l)$; equality is the palindrome condition. Preprocess $O(n)$, query $O(1)$. Test length one, even/odd palindrome, and an off-by-one reverse boundary.

7.3 C. Anagram Substrings

For lowercase input, use 26 prefix counts, not a randomized hash. Two ranges are anagrams iff every character-count difference agrees. Preprocess $O(26n)$ or 26 tables and query $O(26)$; test repeated letters, unequal counts, and full ranges. This is exact and avoids collision reasoning.

In C++ use long long modular multiplication; Python should avoid slicing in the fast path. Stress all three with direct string comparisons on short random data.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.